Critic-Matched Movie Prediction

All-time random holdouts with analytic, boosted-tree, and neural modes

Portfolio project — technical report

Rotten Tomatoes critic reviews (Kaggle, ~1.4M rows through early 2023)

Abstract

We simulate user histories from critics instead of restricting evaluation to a late temporal window. For each fake user, sampled ratings are seen movies and a different rated movie is an unseen target. The analytic formula centers on the target movie's critic mean, applies signed Pearson similarity, and maps critic rating magnitude to the user. Evaluation is *paired and nested*: a fixed target and a fixed seen-order per pseudo-user are reused across every seen-count, so the seen-independent baselines are exactly flat and the curves are smooth. Across 41,472 paired episodes from 556 test critics, full-history analytic RMSE is 0.814 versus 0.827 for the reviewer mean and 0.841 for a calibrated Tomatometer. Two trained models — an XGBoost regressor (0.791) and a deeper residual neural-network ensemble (0.793) — beat every baseline at every seen-count and are statistically tied with each other. The gain is calibration, not taste matching: both models lean first on the movie's Tomatometer, then the user's mean.

1 Data and score scale

The source is `andrezaza/clapper-massive-rotten-tomatoes-movies-and-reviews`. Fractions, letter grades, percentages, and star ratings are parsed and quantized to $\{0, 1, 2, 3, 4, 5\}$, called the *standardized raw score*. It is the prediction target. Critics with at most five parseable scores are excluded, name variants are deduplicated, and repeat critic–movie cells are averaged.

Data funnel	**Count**
Raw review rows (start)	1,444,963
Distinct critics with a parseable score	8,623
After the low-volume critic floor (>5 scored reviews)	4,393 critics, 1,262,747 rows
Parsed, standardized scored reviews used for modelling	992,954
Distinct movies with any scored review	58,736
Neighbour pool / pseudo-users (≥ 10 distinct movies)	3,704 critics
Movies in the final app catalog (most-reviewed)	1,000
Critics appearing in that catalog	3,387

Table 1: From 1.4M raw rows to the modelling pool and the app catalog.

The data retain a diagnostic z-score:

$$z_{c,i} = \frac{r_{c,i} - \mu_c}{\sigma_c}. \tag{1}$$

Here $r_{c,i}$ is critic c's standardized raw score for movie i; μ_c is critic c's all-time mean score; and σ_c is critic c's all-time standard deviation. Thus $z_{c,i} = 1$ means one critic-specific standard deviation above the critic's usual score. Active models use raw scores, not z-scores, because rating magnitude is part of the analytic formula.

The *Tomatometer* is the percentage of reviews labelled Fresh, not a mean 0–5 rating. Baseline B2 fits $t_m = aT_m + b$, where T_m is movie m's Tomatometer percentage, a is a fitted slope, b is a fitted intercept, and t_m is the mapped 0–5 score.

2 All-time random-holdout pseudo-users

The all-time matrix has 3,715 critics with at least 10 scored reviews; 3,704 have 10 distinct movies and can form episodes (the floor was lowered from 20 to widen the pool). Critic identities are split into 2,592 train, 556 validation, and 556 test critics. No trained model sees a validation or test critic as a pseudo-user during fitting.

Training episodes are unpaired for variety: sample n of pseudo-user u's rated movies as seen ratings, then choose a different rated movie as target m (at least three other reviews; u excluded from target aggregates), for $n \in \{3, 5, 10, 20, 50, \text{all}\}$.

Test episodes are *paired and nested* so the comparison across seen-counts is clean. Sampling a fresh random target per n made even the seen-independent baselines wobble column to column. Instead, for each test pseudo-user with more than 50 movies, 8 targets are fixed once; for each target and each of 3 redraws, the remaining movies are placed in one popularity-weighted order, and the seen set at seen-count n is the first n of that order (all for $n = \text{all}$). A single deterministic routine yields these 41,472 episodes, and every model is scored on byte-identical (u, m, n, draw) keys. Because the target and seen order are fixed across n, every seen-count and every baseline is scored on the same films.

This is item-holdout evaluation, not chronological forecasting: a target can predate a seen movie. It enables catalog prediction for unseen movies but does not claim to predict unreleased films.

3 Pearson similarity and magnitude

Let $O_{u,c}$ be movies rated by pseudo-user u and critic c among the sampled seen movies. Pearson correlation is

$$\rho_{u,c} = \frac{\sum_{i \in O_{u,c}} (r_{u,i} - \bar{r}_{u,O})(r_{c,i} - \bar{r}_{c,O})}{\sqrt{\sum_{i \in O_{u,c}} (r_{u,i} - \bar{r}_{u,O})^2} \sqrt{\sum_{i \in O_{u,c}} (r_{c,i} - \bar{r}_{c,O})^2}}. \tag{2}$$

In Equation 2, i indexes a shared seen movie; $r_{u,i}$ and $r_{c,i}$ are user and critic raw scores; $\bar{r}_{u,O}$ and $\bar{r}_{c,O}$ are their respective means over $O_{u,c}$; and $\sum$ adds over the overlap. $\rho_{u,c}$ ranges from -1 for opposite score movement to $+1$ for matching movement. The square roots normalize covariance by each overlap score spread.

Small overlaps are shrunk toward zero:

$$s_{u,c} = \rho_{u,c} \frac{\min(n_{u,c}, k)}{k}. \tag{3}$$

Here $s_{u,c}$ is signed shrunken similarity, $n_{u,c} = |O_{u,c}|$ is overlap count, min selects the smaller input, and validation selected $k = 8$. $|s_{u,c}|$ is absolute, nonnegative similarity magnitude.

The rating-magnitude multiplier is

$$g_{u,c} = \frac{\sum_{i \in O_{u,c}} r_{u,i} r_{c,i}}{\sum_{i \in O_{u,c}} r_{c,i}^2}. \tag{4}$$

$g_{u,c}$ is `mag_sim` in code. It is a least-squares through-origin mapping from critic to user ratings. If $r_{u,i} = 1.25 r_{c,i}$ on every overlap movie, then $g_{u,c} = 1.25$.

4 Movie-mean-centered analytic prediction

For target movie m, let C_m be other critics who rated it. Its movie mean is

$$\bar{r}_m = \frac{\sum_{c \in C_m} r_{c,m}}{|C_m|}. \tag{5}$$

Here $r_{c,m}$ is critic c's target score, $|C_m|$ is the other-reviewer count, and $\bar{r}_m$ is the mean rating for movie m, not a career mean or Tomatometer percentage.

The requested prediction is

$$\hat{r}_{u,m} = \frac{\sum_{c \in C_m} \left(|s_{u,c}|\bar{r}_m + s_{u,c}(r_{c,m} - \bar{r}_m)\right) g_{u,c}}{\sum_{c \in C_m} |s_{u,c}|}. \tag{6}$$

In Equation 6, $\hat{r}_{u,m}$ is the predicted 0–5 score; $s_{u,c}$ comes from Equation 3; $g_{u,c}$ comes from Equation 4; and movie terms come from Equation 5. The denominator intentionally contains only $|s_{u,c}|$, not $g_{u,c}$. Predictions are clipped to $[0, 5]$; a zero denominator falls back to $\bar{r}_m$.

5 Learned models and metrics

Two models share one 38-column feature set, and *both use all of it*: for each target reviewer, the reviewer score minus that critic's leave-one-target all-time mean; reviewers ranked by similarity and their similarity-times-deviation values summarized by mean, count, and standard deviation in ten deciles; then a tail of seen count, overlap statistics, mapped Tomatometer, reviewer count, dispersion, genre, and user mean.

Design 2 is an XGBoost regressor; it handles a missing Tomatometer natively. *Design 3* is a residual neural network in PyTorch, trained on the Apple GPU (MPS). The 37 numeric features (log1p-compressed where they are counts, NaN-imputed and standardized on train statistics) and a 24-dimensional genre embedding feed a projection into six pre-activation residual blocks of width 512 (≈3M weights; batch-norm, GELU, dropout), then a linear head. It is trained with AdamW, a ReduceLROnPlateau schedule, and early stopping; three independently seeded networks are averaged into an ensemble, over 32 fake-user profiles per critic and seen-count (≈437k rows). Full-history training rows and the log compression are both needed so the net does not extrapolate at $n =$ all, where the seen count far exceeds any finite-n training row; the trees are immune to this. Both are verified leakage-free: peer means are leave-one-out and u is excluded everywhere. The same feature code powers training and app inference.

B1 is the all-time global mean; B2 is mapped Tomatometer with reviewer-mean fallback; B3 is $\bar{r}_m$; B4 is an unweighted mean of up to ten positively aligned reviewers. Primary error is

$$\text{RMSE} = \sqrt{\frac{\sum_{j=1}^{N} (\hat{r}_j - y_j)^2}{N}}, \tag{7}$$

where j indexes an episode, $\hat{r}_j$ is a prediction, y_j is its held-out score, and N is episode count. Lower values are better.

6 Results

Seen ratings n	3	5	10	20	50	all
B1 global mean	1.054	1.054	1.054	1.054	1.054	1.054
B2 Tomatometer → score	0.841	0.841	0.841	0.841	0.841	0.841
B3 reviewer mean	0.827	0.827	0.827	0.827	0.827	0.827
B4 top-10 similar mean	0.891	0.892	0.877	0.865	0.859	0.849
Design 1 formula	0.957	0.925	0.869	0.842	0.825	0.814
Design 2 XGBoost	0.815	0.811	0.805	0.801	0.796	**0.791**
Design 3 neural net	0.820	0.816	0.809	0.804	0.798	0.793

Table 2: RMSE on the 41,472 paired, nested episodes. Baselines are flat because they do not depend on the seen set.

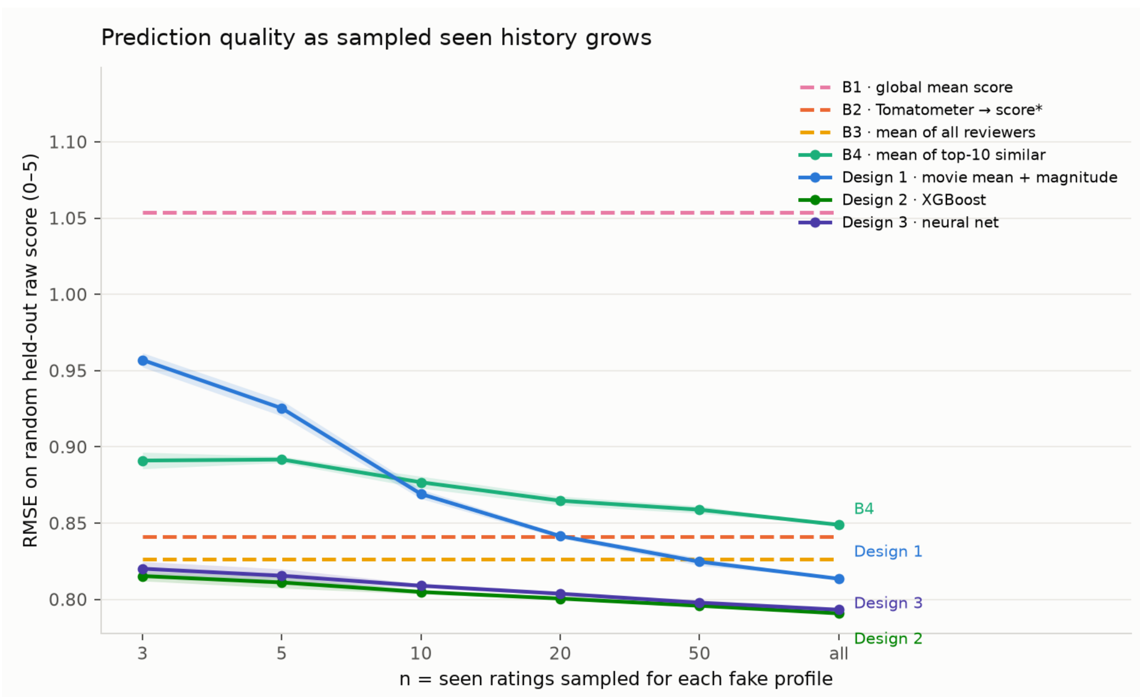

Figure 1: Error versus sampled seen history. With paired evaluation the baselines are exactly flat; Design 1 falls smoothly from 0.957 to 0.814 and crosses them near $n = 50$. Both trained models beat every baseline at every seen-count and overlap each other.

Design 1 improves over B3 by 1.6% and B2 by 3.3% at full history, but the two trained models win everywhere: XGBoost 0.803 overall / 0.791 full history, the neural net 0.807 / 0.793. Scaling the network up — wider (512), deeper (six residual blocks), longer training, and doubled fake-user training data — to look for a double-descent gain moved the loss by nothing, and XGBoost itself was unchanged by the extra data: on this engineered tabular feature set the ceiling is the information in the features, not model capacity. Both models improve only gently with n because their dominant feature is the movie's Tomatometer ($\approx 11\times$ the next feature in XGBoost gain), then the user's mean; this is why the learned loss is as low as it is, and it was checked to be a leakage-free calibrated consensus, not memorization.

Formula component (full-history episodes)	RMSE
Reviewer mean / movie consensus	0.850
Magnitude-scaled movie consensus	**0.833**
Movie-centered signed similarity ($g_{u,c} = 1$)	0.867
Full Design 1 formula	0.850

Table 3: Formula attribution: magnitude scaling supplies the whole gain over consensus.

Magnitude matching supplies the entire improvement over the consensus; the signed similarity term adds nothing on top. As with the trained models, correlation-based taste matching contributes little — the useful personalization is scale/level calibration. Across low, middle, and high critic-disagreement terciles, full-history gain over B2 is +5.1/+4.6/+1.3% for Design 1, +7.1/+6.1/+5.1% for XGBoost, and +7.7/+5.4/+4.8% for the neural net — all three help in every tercile, the trained models most.

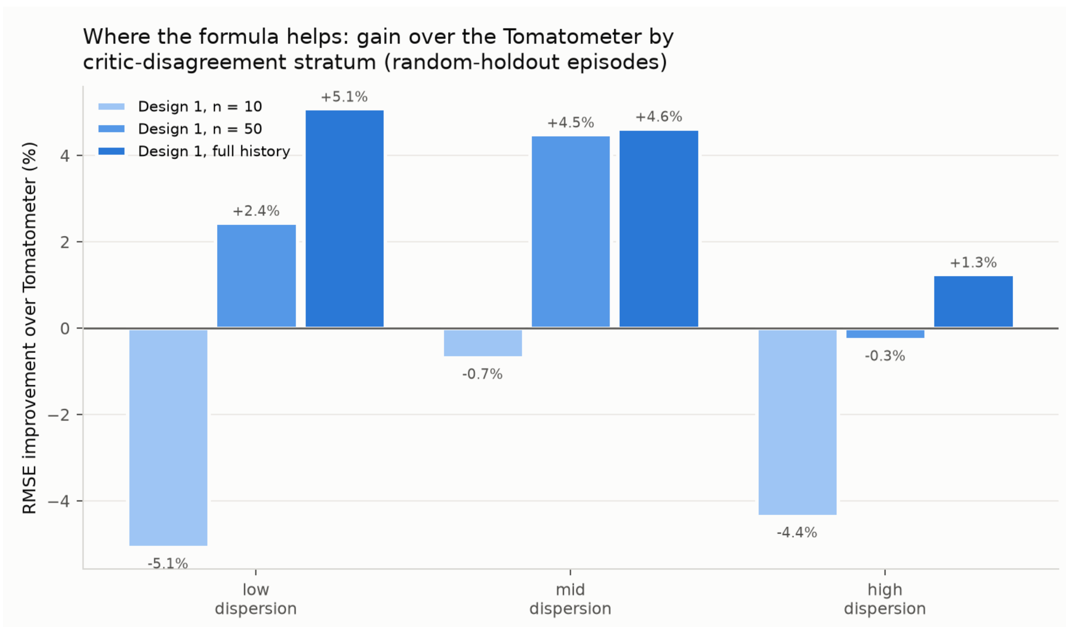

Figure 2: Design 1 gain over B2 by target-movie critic disagreement.

7 Application and limitations

The Streamlit app (Table 1: 1,000 catalog movies, 3,387 critics) is organized in three sections, and a sort control orders the search lists alphabetically (default), by year, or by most reviewed. *(1) Films you have seen*: a search box adds a film to a table where each row carries a five-star widget — click a star and it and all before it light gold, with the rating shown beside it — and the search box clears after each add so films accumulate; rows can be removed. *(2) Films to predict*: the same searchable star-table, but scoring is optional. *(3) Predictions*: for every target film the app shows all three model predictions side by side with the movie's consensus mean and its Tomatometer. When the visitor scores some target films, the app reports the mean squared error of each method — and of the consensus mean — *against the visitor's own score*, and marks the closest, answering “which method predicts me best?”. Predictions update live as stars change.

Item holdout is not temporal forecasting. Lowering the critic floor to 10 adds noisier low-volume critics. The magnitude multiplier is an unregularized through-origin slope. The neural net matches but does not beat the trees here. Scores are quantized to six levels, pseudo-users are professional critics rather than casual users, and Tomatometer values are current snapshots.

Reproducibility. Python, pandas, scipy.sparse, NumPy, XGBoost, PyTorch, matplotlib, and Streamlit. Each design is a self-contained package under `src/` (`design1_analytic`, `design2_xgboost`, `design3_neural`); the pseudo-user substrate and feature code are duplicated verbatim in each, so a reader studies one design without chasing shared modules, and the shared seeds keep the cross-design comparison exact. Run the ordered module pipeline listed in the README from `src/`; all random seeds are fixed.